

O Poder das Funções na Lógica de Programação

Instrutor: Djalma Batista Barbosa Junior
E-mail: djalma.batista@fiemg.com.br



Um Olhar sobre o Código

Antes de entendermos a solução, precisamos sentir a dor do problema. Código com lógica duplicada é difícil de ler, propenso a erros e um pesadelo para modificar. Se uma regra de cálculo muda, temos que alterá-la em vários lugares.

```
ALGORITMO "CalculoDeMedias"
VAR
    nota1_alunoA, nota2_alunoA, media_alunoA: REAL
    nota1_alunoB, nota2_alunoB, media_alunoB: REAL
    nota1_alunoC, nota2_alunoC, media_alunoC: REAL
INICIO
    // Lógica para o Aluno A
    LEIA(nota1_alunoA, nota2_alunoA)
    media_alunoA <- (nota1_alunoA + nota2_alunoA) / 2
    ESCREVA("Média do Aluno A: ", media_alunoA)

    // A MESMA LÓGICA para o Aluno B
    LEIA(nota1_alunoB, nota2_alunoB)
    media_alunoB <- (nota1_alunoB + nota2_alunoB) / 2
    ESCREVA("Média do Aluno B: ", media_alunoB)

    // E DE NOVO para o Aluno C...
    LEIA(nota1_alunoC, nota2_alunoC)
    media_alunoC <- (nota1_alunoC + nota2_alunoC) / 2
    ESCREVA("Média do Aluno C: ", media_alunoC)
FIM_ALGORITMO
```

A Solução: O Conceito de Modularização

A Modularização é a técnica de dividir um algoritmo grande em blocos menores, independentes e reutilizáveis, chamados de sub-rotinas (ou funções). Cada bloco tem uma responsabilidade única e bem definida.

Em Pseudolinguagem, temos dois tipos de sub-rotinas:

- Procedimento: Um bloco de código que realiza uma ação (ex: exibir uma mensagem, limpar a tela). Ele executa uma tarefa, mas não devolve um resultado formal. Função:
- Um bloco de código que, além de realizar uma ação, retorna um valor (ex: calcular uma média, validar um CPF). Ela sempre produz uma resposta. Reflexão: Como o código do slide anterior poderia ser simplificado se tivéssemos um "módulo" chamado CalcularMedia que pudéssemos usar três vezes?

Anatomia de um PROCEDIMENTO

Um Procedimento é ideal para tarefas que não precisam de uma resposta direta. Pense nele como um comando que você dá ao programa.

```
// --- Definição do Procedimento ---
PROCEDIMENTO ExibirResultado(nomeAluno: CARACTERE, mediaFinal: REAL)
//   ^Palavra-Chave   ^Nome       ^Parâmetros de Entrada
INICIO
    // --- Bloco de Comandos (O que ele faz) ---
    ESCREVAL("-----")
    ESCREVAL("Boletim do Aluno: ", nomeAluno)
    ESCREVAL("Média Final: ", mediaFinal)
    ESCREVAL("-----")
FIMPROCEDIMENTO // Fim da definição

// --- Como chamar o Procedimento no algoritmo principal ---
ALGORITMO "Principal"
INICIO
    ExibirResultado("João", 7.5)
    ExibirResultado("Maria", 9.0)
FIM_ALGORITMO
```

Anatomia de uma FUNÇÃO

Uma Função é essencial quando você precisa de um cálculo ou de uma verificação que gere um resultado para ser usado em outra parte do seu código.

```
// --- Definição da Função ---  
FUNCAO CalcularMedia(nota1: REAL, nota2: REAL) : REAL  
// ^Palavra-Chave ^Nome          ^Parâmetros      ^Tipo de Retorno  
VAR  
    mediaCalculada: REAL // Variável local  
INICIO  
    // --- Bloco de Comandos (O que ela faz) ---  
    mediaCalculada <- (nota1 + nota2) / 2  
    RETORNE mediaCalculada // O "coração" da função: a resposta  
FIMFUNCAO // Fim da definição  
  
// --- Como usar a Função no algoritmo principal ---  
ALGORITMO "Principal"  
VAR  
    mediaTurma: REAL  
INICIO  
    // O retorno da função é atribuído a uma variável  
    mediaTurma <- CalcularMedia(8.0, 9.0)  
    ESCREVA("A primeira média calculada foi: ", mediaTurma)  
FIM_ALGORITMO
```

A Via de Comunicação

Parâmetros são variáveis especiais declaradas na assinatura de uma função/procedimento que servem para receber dados do mundo exterior. Eles tornam a função genérica e flexível.

Parâmetro vs. Argumento:

- Parâmetro: A variável na definição da função. É o "molde".
- Argumento: O valor real passado na chamada da função. É o "recheio".

```
// "valor" e "percentual" são os PARÂMETROS
FUNCAO CalcularAumento(valor: REAL, percentual: REAL) : REAL
VAR
    valorAumento: REAL
INICIO
    valorAumento <- valor * (percentual / 100)
    RETORNE valor + valorAumento
FIMFUNCAO
```

```
ALGORITMO "CalculaSalario"
VAR
    salarioAntigo, novoSalario: REAL
INICIO
    salarioAntigo <- 2000.00
    // 2000.00 e 10.0 são os ARGUMENTOS passados para a função
    novoSalario <- CalcularAumento(salarioAntigo, 10.0)
    ESCREVA("O novo salário é: ", novoSalario)
FIM_ALGORITMO
```

O Comando Retorne: A Resposta da Função

Conceito: A palavra-chave RETORNE tem duas funções cruciais:

- Ela finaliza a execução da função imediatamente.
- Ela envia um valor de volta para o local onde a função foi chamada.

Como o Retorno Funciona:

Uma chamada de função em uma expressão é "substituída" pelo seu valor de retorno quando o código é executado.

```
FUNCAO EhMaiorDeIdade(idade: INTEIRO) : LOGICO
INICIO
    SE idade >= 18 ENTAO
        RETORNE VERDADEIRO // Encerra a função e devolve VERDADEIRO
    SENAO
        RETORNE FALSO // Encerra a função e devolve FALSO
    FIMSE
    // Nenhum código aqui seria executado, pois o RETORNE já encerrou a função
FIMFUNCAO

ALGORITMO "VerificaAcesso"
VAR
    podeEntrar: LOGICO
INICIO
    // A chamada EhMaiorDeIdade(25) se torna o valor VERDADEIRO
    podeEntrar <- EhMaiorDeIdade(25)

    SE podeEntrar = VERDADEIRO ENTAO
        ESCREVA("Acesso liberado!")
    FIMSE
FIM_ALGORITMO
```

Escopo de Variáveis: Onde Cada Variável "Mora"?

O escopo define a "vida útil" e a visibilidade de uma variável. Entender isso é crucial para evitar bugs e criar funções independentes.

Tipos de Escopo:

- Global: Declarada fora de qualquer função. Pode ser acessada e modificada por qualquer parte do algoritmo. (Use com moderação!)
- Local: Declarada dentro de uma função/procedimento. Só existe e pode ser acessada dentro daquela função. Ela é criada quando a função começa e destruída quando termina.

```
ALGORITMO "TesteDeEscopo"
VAR
    saldoGeral: REAL // <- Variável GLOBAL

PROCEDIMENTO RealizarCompra(valorCompra: REAL)
VAR
    imposto: REAL // <- Variável LOCAL, só existe aqui dentro
INICIO
    imposto <- valorCompra * 0.10
    saldoGeral <- saldoGeral - valorCompra - imposto // Acessando a global
    ESCREVA("Compra realizada. Saldo restante: ", saldoGeral)
FIMPROCEDIMENTO

INICIO // Algoritmo Principal
    saldoGeral <- 1000.00
    RealizarCompra(200.00)
    // A linha abaixo daria ERRO, pois "imposto" não existe aqui!
    // ESCREVA("O imposto da compra foi: ", imposto)
FIM_ALGORITMO
```


Vantagens e Boas Práticas

- Reutilização: Escreva uma vez, use quantas vezes precisar.
- Legibilidade: O código principal se torna uma sequência de chamadas, muito mais fácil de ler.
- Manutenção: Precisa corrigir um bug no cálculo de juros? Altere apenas na Funcao CalcularJuros, e a correção se aplica a todo o sistema.
- Abstração: Você pode usar uma função sem precisar saber como ela funciona por dentro.

Boas Práticas:

- Responsabilidade Única: Uma função deve fazer uma única coisa e fazê-la bem.
- Nomes Descritivos: O nome da função deve ser um verbo que descreve sua ação (ValidarEmail, CalcularImposto).
- Poucos Parâmetros: Funções com muitos parâmetros (mais de 3 ou 4) podem ser um sinal de que elas têm responsabilidades demais.

Exemplo Prático: Um Algoritmo Modular Completo

Cenário:

Ler duas notas de um aluno, validar se estão no intervalo correto (0-10), calcular a média e exibir o status (Aprovado/Reprovado).

```
// Função para ler e validar uma única nota
FUNCAO LerNotaValida() : REAL
VAR nota: REAL
INICIO
    REPITA
        ESCREVA("Digite uma nota entre 0 e 10: ")
        LEIA(nota)
    ATE (nota >= 0) E (nota <= 10)
    RETORNE nota
FIMFUNCAO

// Função para determinar o status do aluno
FUNCAO ObterStatus(media: REAL) : CARACTERE
INICIO
    SE media >= 7.0 ENTAO
        RETORNE "Aprovado"
    SENAO
        RETORNE "Reprovado"
    FIMSE
FIMFUNCAO
```

```
ALGORITMO "SistemaDeNotasModular"
```

```
VAR
```

```
    nota1, nota2, mediaFinal: REAL
```

```
    statusAluno: CARACTERE
```

```
INICIO
```

```
    ESCREVAL("--- Cadastro de Notas ---")
```

```
    nota1 <- LerNotaValida() // 1ª chamada
```

```
    nota2 <- LerNotaValida() // 2ª chamada (Reutilização!)
```

```
    mediaFinal <- (nota1 + nota2) / 2
```

```
    statusAluno <- ObterStatus(mediaFinal)
```

```
    ESCREVAL("A média do aluno é ", mediaFinal, ". Status: ", statusAluno)
```

```
FIM_ALGORITMO
```

Funções: Seus Blocos de Construção

- Funções combatem a repetição e trazem organização.
- Procedimentos executam ações; Funções calculam e retornam resultados.
- Parâmetros são as entradas; Retorno é a saída.
- O escopo de variáveis (global vs. local) é um conceito fundamental para o isolamento e a segurança do seu código.
- Pensar de forma modular é o primeiro passo para criar algoritmos complexos e de fácil manutenção.

AGRADECEMOS SUA PARTICIPAÇÃO

